

Strategy Context breakpoint explanation

Where the Context is, why not to put the breakpoint on the class line, and how to prove the Strategy sequence diagram at runtime.

Where is the Context?

The Strategy Context is the class **SeparationEngine**. It is located in **app/ml/separation_engine.py**.

```
15: class SeparationEngine:
```

This class is the Context because it owns the selected algorithm reference and runs separation through that reference.

```
18: algorithm: SeparationAlgorithm
```

The important design point is that the Context stores the algorithm as SeparationAlgorithm, not as a hardcoded NeoSSNet-only object.

Can I put a breakpoint on the class line?

You can, but it is not useful for the demo. A breakpoint on class SeparationEngine may trigger when Python imports the file during server startup. That only proves the class was loaded, not that the user workflow reached the Strategy Pattern.

For the lecturer, the better proof is a breakpoint inside the method that runs during separation:

```
29: return self.algorithm.separate()
```

This line is inside the SeparationEngine class. It proves the Context is actually running during the Run Separation request.

Why this matches the sequence diagram

Sequence diagram step	Runtime proof	Why this is correct
SeparationEngine (Context)	app/ml/separation_engine.py:29	Execution is inside the Context class method.
SeparationAlgorithm (Strategy)	self.algorithm.separate(...)	self.algorithm is typed as SeparationAlgorithm, so the Context depends on the Strategy contract.
NeoSSNetStrategy (ConcreteStrategy)	Step Into reaches app/ml/neossnet_strategy.py:23	Python dynamic dispatch sends the call to the real runtime object.

Recommended runtime breakpoints

Use these breakpoints to prove the Strategy Pattern cleanly during the live demo.

#	Purpose	File and line	Code
1	Context object is created and receives algorithm	app/services/separation_service.py:138	engine = self.engine_class(algorithm=algorithm, device_name="cpu")
2	Context calls Strategy	app/ml/separation_engine.py:29	return self.algorithm.separate()
3	ConcreteStrategy runs	app/ml/neossnet_strategy.py:23	if model_config_path is None:

Step-by-step demo

Step 1: Click Run Separation. Stop at **separation_service.py:138**. Inspect **algorithm**. It should be **NeoSSNetStrategy**.

Step 2: Step Over line 138. Inspect **engine** and **engine.algorithm**. This proves SeparationEngine was created with the selected strategy object.

Step 3: Continue to **separation_engine.py:29**. Now execution is inside the Context. This is the main Context proof.

Step 4: Press **Step Into** on line 29. VS Code should enter **neossnet_strategy.py:23**. This proves the ConcreteStrategy runs.

Why the Strategy interface does not stop as a separate runtime line

SeparationAlgorithm is a Python Protocol interface in **app/ml/separation_algorithm.py**. It defines the method contract, but it does not contain the real separation logic. Because of that, Step Into usually moves from the Context directly to the ConcreteStrategy.

```
22: class SeparationAlgorithm(Protocol):
25: def separate(
```

This still proves the sequence diagram because the call is made through **self.algorithm**, which is the Strategy abstraction, and the runtime object is **NeoSSNetStrategy**.

What to say to the lecturer

Short speaking script for the Strategy Context proof.

- Now I will show the Strategy Pattern using the actual runtime code.
- The Context is the SeparationEngine class in app/ml/separation_engine.py.
- I do not put the breakpoint on class SeparationEngine, because that line runs when Python imports the file, not when the user clicks Run Separation.
- Instead, I put the important breakpoint at separation_engine.py line 29, inside the Context method.
- At this line, SeparationEngine calls self.algorithm.separate(). This proves the Context uses the Strategy abstraction instead of calling NeoSSNet directly.
- When I Step Into, execution enters NeoSSNetStrategy. That proves the concrete strategy selected at runtime is NeoSSNetStrategy.
- This matches the sequence diagram: SeparationEngine calls SeparationAlgorithm, and the runtime implementation is NeoSSNetStrategy.
- The benefit is maintainability. A future algorithm can implement the same interface, and SeparationEngine can run it without changing the engine workflow.

If the browser waits

Say: "The browser is waiting because VS Code paused the FastAPI request at the backend breakpoint. When I press Continue, the backend response returns and the page updates."

One-line proof

Best proof line: **app/ml/separation_engine.py:29** because it is inside the Context and it calls the Strategy object.